



INSTITUTO POLITECNICO NACIONAL
Unidad Profesional Interdisciplinaria en
Ingeniería y Tecnologías Avanzadas



BASES DE DATOS DISTRIBUIDAS

PRACTICA 1



ALUMNOS:

Fuentes García Liliana Alejandra

Marcelino Ceron Diego Antonio

GRUPO: 3TM2

1.1

```
select name, cant
from Production.product p
join (select top 10 productid, sum(orderqty) cant
      from sales.SalesOrderDetail sod
      join sales.SalesOrderHeader soh
      on sod.SalesOrderID = soh.SalesOrderID
      where year(soh.OrderDate)=2014
      group by productid
      order by cant desc) as T
on p.ProductID = t.ProductID
```

	name	cant
1	Water Bottle - 30 oz.	2902
2	AWC Logo Cap	1950
3	Sport-100 Helmet, Blue	1776
4	Patch Kit/8 Patches	1761
5	Sport-100 Helmet, Red	1717
6	Sport-100 Helmet, Black	1716
7	Mountain Tire Tube	1501
8	Classic Vest, S	1395
9	Short-Sleeve Classic Jersey, XL	1264
10	Road Tire Tube	1156

Se opto por usar una subconsulta(derivada)y crear una tabla temporal para después hacer un join con la tabla que contiene la información del producto. En la subconsulta se ocuparon 2 tablas para checar los productos mas vendidos con la restricción de que sean del año 2014, para ello se uso un *top 10,sum(orderqty)* y la condicion *year(soh.orderID)*

1.2

```
SELECT p.Name,  
       T.cant,  
       T.PrecioPromedio  
FROM Production.Product p  
JOIN (  
    SELECT TOP 10  
           sod.ProductID,  
           SUM(sod.OrderQty) AS cant,  
           AVG(sod.UnitPrice) AS PrecioPromedio  
    FROM Sales.SalesOrderDetail sod  
    JOIN Sales.SalesOrderHeader soh  
    ON sod.SalesOrderID = soh.SalesOrderID  
    JOIN Production.Product p  
    ON sod.ProductID = p.ProductID  
    WHERE YEAR(soh.OrderDate)=2014  
    AND p.ListPrice > 1000  
    GROUP BY sod.ProductID  
    ORDER BY cant DESC  
    ) AS T  
ON p.ProductID = T.ProductID;
```

Results		Messages	
	Name	cant	PrecioPromedio
1	Mountain-200 Black, 38	619	1981.7872
2	Road-350-W Yellow, 48	586	1388.3549
3	Road-350-W Yellow, 40	530	1391.601
4	Mountain-200 Black, 42	513	2058.144
5	Mountain-200 Silver, 38	508	2079.2043
6	Mountain-200 Silver, 42	441	2086.7745
7	Mountain-200 Black, 46	437	2050.3074
8	Mountain-200 Silver, 46	432	2095.4693
9	Touring-1000 Yellow, 60	397	1812.1551
10	Touring-1000 Blue, 60	393	1839.7073

En este caso, el código fue modificando, tanto la tabla temporal(subconsulta) como la consulta, en el primero se agregando varias condiciones como el *listprice* fuera mayor que 1000 y que se calculara el valor promedio usando AVG(), para que sql mostrara información, se agregaron nuevos atributos en la consulta para ser coherentes con la información que arroja la subconsulta y pueda mostrarla

2.1.-

- Consulta realizada.

```
SELECT TOP 10
    e.BusinessEntityID,
    p.FirstName,
    p.LastName,
    SUM(soh.SubTotal) AS TotalVentas
FROM HumanResources.Employee e
JOIN Person.Person p
    ON p.BusinessEntityID = e.BusinessEntityID
JOIN Sales.SalesOrderHeader soh
    ON soh.SalesPersonID = e.BusinessEntityID
JOIN Sales.SalesTerritory st
    ON st.TerritoryID = soh.TerritoryID
WHERE st.Name = 'Northwest'
GROUP BY e.BusinessEntityID, p.FirstName, p.LastName
HAVING
    SUM(soh.SubTotal) >
    (
        -- Promedio de ventas por empleado en Northwest
        SELECT AVG(VentasEmpleado)
        FROM (
            SELECT
                soh2.SalesPersonID,
                SUM(soh2.SubTotal) AS VentasEmpleado
            FROM Sales.SalesOrderHeader soh2
            JOIN Sales.SalesTerritory st2
                ON st2.TerritoryID = soh2.TerritoryID
            WHERE st2.Name = 'Northwest'
                AND soh2.SalesPersonID IS NOT NULL
            GROUP BY soh2.SalesPersonID
        ) AS t
    )
ORDER BY TotalVentas DESC;
```

- Resultado obtenido.

	BusinessEntityID	FirstName	LastName	TotalVentas
1	283	David	Campbell	3729945,349
2	280	Pamela	Ansman-Wolfe	3325102,5941
3	284	Tete	Mensa-Annan	2312545,69
4	276	Linda	Mitchell	2084593,1619

- Conclusión.

La consulta obtiene a los empleados que han vendido más que el promedio de ventas por empleado en el territorio Northwest. Para hacerlo, primero une las tablas de empleados, personas y órdenes de venta, de modo que pueda relacionar a cada empleado con su nombre completo y con las ventas que realizó.

Luego se filtran únicamente las órdenes que pertenecen al territorio Northwest. Una vez hecho esto, la consulta agrupa los datos por empleado para calcular el total de ventas de cada uno mediante la función SUM.

La parte clave es el HAVING, donde se compara el total de ventas de cada empleado contra un subquery que calcula el promedio de ventas de todos los empleados en Northwest. Ese subquery obtiene primero las ventas totales por cada vendedor en esa zona, y luego calcula el promedio de esos totales.

Finalmente, solo se muestran los empleados cuyo total de ventas supera ese promedio, y el resultado se ordena de mayor a menor, mostrando únicamente los empleados que vendieron más que el promedio .

2.2.- Una vez resuelta la consulta convierte la subconsulta en un CTE (Common Table Expresión).

- Consulta realizada.

```

WITH VentasNorthwest AS (
    SELECT
        soh.SalesPersonID,
        SUM(soh.SubTotal) AS VentasEmpleado
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesTerritory st
        ON st.TerritoryID = soh.TerritoryID
    WHERE st.Name = 'Northwest'
        AND soh.SalesPersonID IS NOT NULL
    GROUP BY soh.SalesPersonID
),
PromedioNW AS (
    SELECT AVG(VentasEmpleado) AS PromedioVentas
    FROM VentasNorthwest
)

SELECT TOP 10
    e.BusinessEntityID,
    p.FirstName,
    p.LastName,
    SUM(soh.SubTotal) AS TotalVentas
FROM HumanResources.Employee e
JOIN Person.Person p
    ON p.BusinessEntityID = e.BusinessEntityID
JOIN Sales.SalesOrderHeader soh
    ON soh.SalesPersonID = e.BusinessEntityID
JOIN Sales.SalesTerritory st
    ON st.TerritoryID = soh.TerritoryID
CROSS JOIN PromedioNW
WHERE st.Name = 'Northwest'
GROUP BY e.BusinessEntityID, p.FirstName, p.LastName, PromedioNW.PromedioVentas
HAVING SUM(soh.SubTotal) > PromedioNW.PromedioVentas
ORDER BY TotalVentas DESC;

```

- Resultado obtenido.

	BusinessEntityID	FirstName	LastName	TotalVentas
1	283	David	Campbell	3729945,349
2	280	Pamela	Ansman-Wolfe	3325102,5941
3	284	Tete	Mensa-Annan	2312545,69
4	276	Linda	Mitchell	2084593,1619

- Conclusiones

La consulta utiliza dos CTE para resolver el problema de listar a los empleados que han vendido más que el promedio de ventas por empleado en el territorio Northwest.

Primero, el CTE VentasNorthwest calcula cuánto vendió cada empleado dentro de Northwest agrupando por SalesPersonID y sumando sus ventas. Luego, el CTE PromedioNW toma esos totales individuales y obtiene el promedio general de ventas de todos los empleados en ese territorio.

En el SELECT principal se unen las tablas de empleados, personas y órdenes de venta para obtener el nombre del empleado y sus ventas realizadas específicamente en Northwest. Con el CROSS JOIN se incorpora el promedio calculado por el CTE.

Después, la consulta agrupa por empleado y calcula su total de ventas dentro del territorio. En el HAVING se compara ese total con el promedio obtenido en el CTE: solo se mantienen los empleados cuyo total de ventas es mayor que el promedio de ventas por empleado en Northwest.

Finalmente, se ordenan los resultados de mayor a menor y se muestran los empleados con mejor desempeño por encima del promedio.

3.1.- Calcula ventas totales por territorio y año, mostrando solo aquellos con más de 5 órdenes y ventas > \$1,000,000, ordenado por ventas descendente.

- Consulta realizada,

```
SELECT
    st.Name AS Territorio,
    YEAR(soh.OrderDate) AS Año,
    COUNT(soh.SalesOrderID) AS NumeroOrdenes,
    SUM(soh.SubTotal) AS VentasTotales
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st
    ON st.TerritoryID = soh.TerritoryID
GROUP BY
    st.Name,
    YEAR(soh.OrderDate)
HAVING
    COUNT(soh.SalesOrderID) > 5
    AND SUM(soh.SubTotal) > 1000000
ORDER BY
    VentasTotales DESC;
```

- Resultado obtenido.

	Territorio	Año	NumeroOrdenes	VentasTotales
1	Southwest	2013	2725	9116540,3149
2	Southwest	2012	777	8288853,8953
3	Canada	2013	1884	6229517,5635
4	Northwest	2013	2053	6015174,736
5	Canada	2012	460	5861057,4737
6	Northwest	2012	510	4735144,1968
7	Canada	2013	2015	4330664,3310

- Conclusión.

En esta consulta primero se selecciona la tabla de órdenes y se utiliza un JOIN para unirla con la tabla de territorios, de manera que podamos obtener el nombre del territorio correspondiente a cada orden. Luego, con GROUP BY, se agrupan los registros por territorio y por año, permitiendo calcular cuántas órdenes se realizaron y cuánto se vendió en total en cada combinación.

Una vez formados los grupos, se aplican filtros mediante HAVING, que sirve para descartar territorios/años con poca actividad; solo se muestran aquellos que superan 5 órdenes y un millón de dólares en ventas. Finalmente, los resultados se ordenan de mayor a menor según el total vendido.

3.2.- Agregar la desviación estándar de ventas

- Consulta realizada.

```
SELECT
    st.Name AS Territorio,
    YEAR(soh.OrderDate) AS Año,
    COUNT(soh.SalesOrderID) AS NumeroOrdenes,
    SUM(soh.SubTotal) AS VentasTotales,
    STDEV(soh.SubTotal) AS DesviacionEstandar
FROM Sales.SalesOrderHeader soh
JOIN Sales.SalesTerritory st
    ON st.TerritoryID = soh.TerritoryID
GROUP BY
    st.Name,
    YEAR(soh.OrderDate)
HAVING
    COUNT(soh.SalesOrderID) > 5
    AND SUM(soh.SubTotal) > 1000000
ORDER BY
    VentasTotales DESC;
```

- Resultado obtenido.

	Territorio	Año	NumeroOrdenes	VentasTotales	DesviacionEstandar
1	Southwest	2013	2725	9116540,3149	11926,7192978251
2	Southwest	2012	777	8288853,8953	20994,8790940681
3	Canada	2013	1884	6229517,5635	11840,7134354727
4	Northwest	2013	2053	6015174,736	11211,2813080162
5	Canada	2012	460	5861057,4737	21417,4889426151

- Conclusión.

En esta consulta la desviación estándar se obtiene tomando todos los subtotales de las órdenes que pertenecen al mismo territorio y al mismo año, definidos previamente por el GROUP BY. SQL calcula automáticamente la desviación estándar usando la fórmula estadística clásica, midiendo qué tanto se alejan los subtotales individuales del promedio de ese grupo. Esto permite conocer si las ventas dentro del territorio/año fueron consistentes o muy variables.

4.1.- Encuentra vendedores que han vendido TODOS los productos de la categoría Bikes.

- Consulta realizada,

```

SELECT
    sp.BusinessEntityID AS SalesPersonID,
    pp.FirstName,
    pp.LastName
FROM Sales.SalesPerson sp
JOIN Person.Person pp
    ON pp.BusinessEntityID = sp.BusinessEntityID
WHERE NOT EXISTS (
    -- Productos de la categoría Bikes que este vendedor NO ha vendido
    SELECT p.ProductID
    FROM Production.ProductCategory pc
    JOIN Production.ProductSubcategory ps
        ON ps.ProductCategoryID = pc.ProductCategoryID
    JOIN Production.Product p
        ON p.ProductSubcategoryID = ps.ProductSubcategoryID
    WHERE pc.Name = 'Bikes'
    AND p.ProductID NOT IN (
        SELECT sod.ProductID
        FROM Sales.SalesOrderHeader soh
        JOIN Sales.SalesOrderDetail sod
            ON sod.SalesOrderID = soh.SalesOrderID
        WHERE soh.SalesPersonID = sp.BusinessEntityID
    )
);

```

- Resultado obtenido.

	SalesPersonID	FirstName	LastName
1	279	Tsvi	Reiter
2	277	Jillian	Carson
3	276	Linda	Mitchell
4	282	José	Saraiva
5	281	Shu	Ito

- Conclusión.

La consulta obtiene a los vendedores y sus nombres, uniendo SalesPerson con Person. Luego aplica un filtro con NOT EXISTS, que revisa si existe algún producto de la categoría Bikes que el vendedor no haya vendido.

Para ello, la subconsulta interna obtiene todos los productos de Bikes y verifica cuáles no aparecen entre los productos que ese vendedor ha vendido según sus órdenes. Si se encuentra aunque sea un solo producto de Bikes que el vendedor no vendió, queda fuera del resultado.

Por lo tanto, la consulta final muestra únicamente a los vendedores que sí han vendido todos los productos de la categoría Bikes.

4.2.- Cambia a categoría "Clothing" (ID=4).

- Consulta realizada

Solo se modifiko esta parte de la consulta anterior

```
WHERE pc.Name = 'Clothing'
AND p.ProductID NOT IN (
    SELECT sod.ProductID
    FROM Sales.SalesOrderHeader soh
    JOIN Sales.SalesOrderDetail sod
    ON sod.SalesOrderID = soh.SalesOrderID
    WHERE soh.SalesPersonID = sp.BusinessEntityID
)
```

- Resultado obtenido.

SalesPersonID	FirstName	LastName
---------------	-----------	----------

- Conclusión.

La consulta lista a los vendedores que han vendido todos los productos de la categoría "Clothing".

Primero une la tabla SalesPerson con Person para obtener el nombre y apellido de cada vendedor.

Luego usa NOT EXISTS para verificar si existe algún producto de la categoría Clothing que el vendedor no haya vendido. Para eso, la subconsulta obtiene todos los productos de esa categoría y revisa cuáles no aparecen entre los productos vendidos por el vendedor según sus órdenes en SalesOrderHeader y SalesOrderDetail.

Si se encuentra al menos un producto de Clothing que el vendedor no haya vendido, entonces queda fuera del resultado. Por eso, solo se muestran los vendedores que sí han vendido absolutamente todos los productos de la categoría Clothing.

5

```

WITH Ventas AS (
SELECT
    pc.Name AS Categoria,
    p.Name AS Producto,
    SUM(sod.OrderQty) AS TotalVendido,
    ROW_NUMBER() OVER(
        PARTITION BY pc.Name
        ORDER BY SUM(sod.OrderQty) DESC
    ) AS rn
FROM AdventureWorks2022.Sales.SalesOrderDetail sod
JOIN SV_SELF.AdventureWorks2022.Production.Product p
    ON sod.ProductID = p.ProductID
JOIN SV_SELF.AdventureWorks2022.Production.ProductSubcategory ps
    ON p.ProductSubcategoryID = ps.ProductSubcategoryID
JOIN SV_SELF.AdventureWorks2022.Production.ProductCategory pc
    ON ps.ProductCategoryID = pc.ProductCategoryID
GROUP BY pc.Name, p.Name
)
SELECT Categoria, Producto, TotalVendido
FROM Ventas
WHERE rn = 1;

```

%		
Results	Messages	
Categoria	Producto	TotalVendido
Accessories	Water Bottle - 30 oz.	6815
Bikes	Mountain-200 Black, 38	2977
Clothing	AWC Logo Cap	8311
Components	LL Road Frame - Black, 52	1581

- Conclusión.

Se utilizó una CTE mediante el uso WITH(subconsulta)para calcular el total de ventas por producto y categoría, después se uso la función de ventana ROW_NUMBER() con PARTITION BY ocasionando numerar los productos dentro de cada categoría según su cantidad vendida en orden descendente, por último, se seleccionaron únicamente los registros con número 1, obteniendo así el producto más vendido por cada categoría utilizando información de los servidores.